Chapitre 1

General Network Basics

Outline

- Revisiting Network Basics
  - Network layers
  - Unicast, Multicast, Broadcast, Anycast
  - Lan-Wan-Man
  - Internet - Intranet - Extranet
  - Tcp/Ip
- Introduction to routers
  - Router or firewall
  - iptables Firewall
  - iptables NAT

Plan

- **Revisiting Network Basics**
  - **Network layers**
  - **Unicast, Multicast, Broadcast, Anycast**
  - **Lan-Wan-Man**
  - **Internet - Intranet - Extranet**
  - **Tcp/Ip**
- Introduction to routers
  - Router or firewall
  - iptables Firewall
  - iptables NAT

3

Seven OSI layers/Four DoD layers

4

Seven OSI layers/Four DoD layers

- **Layer 1**: is all about voltage, electrical signals and connections.
  - Devices like **repeaters** and **hubs** are part of this layer and are amplifying electrical signals on cables.

OSI Model	DoD Model	protocols		devices/apps
layer 5, 6, 7	application	dns, dhcp, ntp, snmp, https, ftp, ssh, telnet, http, pop3... others		web server, mail server, browser, mail client...
layer 4	host-to-host	tcp	udp	gateway
layer 3	internet	ip, icmp, igmp		router, firewall layer 3 switch
layer 2	network access	arp (mac), rarp		bridge layer 2 switch
layer 1		ethernet, token ring		hub

5

Seven OSI layers/Four DoD layers

- **Layer 2**: is about frames. A frame has a crc (cyclic redundancy check).
  - Each network card is identifiable by a unique **48-bit mac address** (media access control address).
  - Devices like **bridges** and **switches**. A bridge is more intelligent than a hub because a bridge can make decisions based on the mac address of computers.
  - **arp** and **ifconfig** to explore this layer.

OSI Model	DoD Model	protocols		devices/apps
layer 5, 6, 7	application	dns, dhcp, ntp, snmp, https, ftp, ssh, telnet, http, pop3... others		web server, mail server, browser, mail client...
layer 4	host-to-host	tcp	udp	gateway
layer 3	internet	ip, icmp, igmp		router, firewall layer 3 switch
layer 2	network access	arp (mac), rarp		bridge layer 2 switch
layer 1		ethernet, token ring		hub

6

Seven OSI layers/Four DoD layers

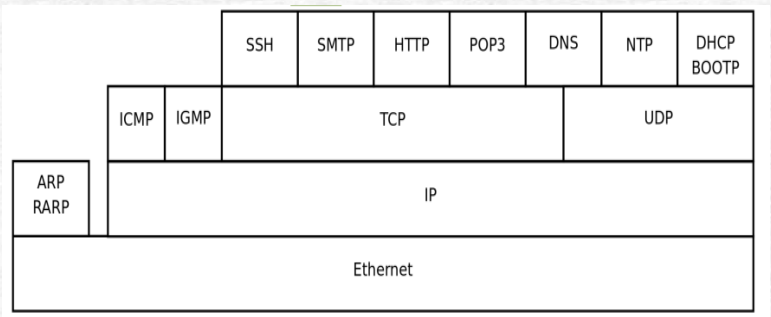
- **Layer 3**: commonly referred to as the **internet layer**, is about ip packets.
  - Gives every host a unique **32-bit ip address.**
  - IP is not the only protocol on this layer, there is also icmp, igmp, ipv6 and more. A complete list can be found in the /etc/protocols file.
  - Devices like **routers** and **layer 3 switches**, are devices that know (and have) an ip address.

OSI Model	DoD Model	protocols		devices/apps
layer 5, 6, 7	application	dns, dhcp, ntp, snmp, https, ftp, ssh, telnet, http, pop3... others		web server, mail server, browser, mail client...
layer 4	host-to-host	tcp	udp	gateway
layer 3	internet	ip, icmp, igmp		router, firewall layer 3 switch
layer 2	network access	arp (mac), rarp		bridge layer 2 switch
layer 1		ethernet, token ring		hub

7

Seven OSI layers/Four DoD layers

- Very common protocols that are discussed in this module.



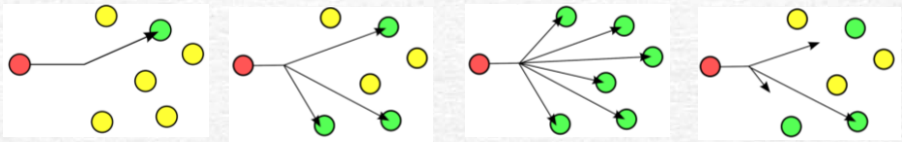
8

Unicast, Multicast, Broadcast, Anycast

9

Unicast, Multicast, Broadcast, Anycast

- A **unicast** communication originates from **one computer** and is destined for exactly **one other computer** (or host). It is common for computers to have many unicast communications.
- A **multicast** is destined for **a group** (of computers), e.g., routers (RIPv2) use it to exchange routing tables, a media server sends one video stream to multiple clients in the same multicast group.
- A **broadcast** is meant for **everyone**.
  - A **layer two broadcast** is received by all network cards on the same segment (it does not pass any router),
  - A **layer 3 broadcast** is received by all hosts in the **same ip subnet**.
- An **anycast** signal goes the (geographically) **nearest** of a well defined group.

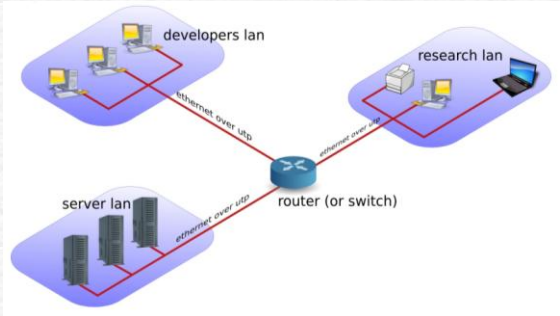


10

Lan-Wan-Man

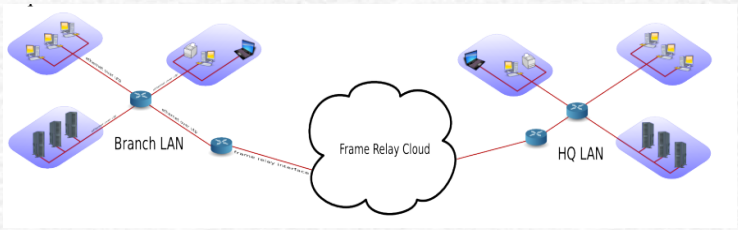
Lan-Wan-Man

- A **LAN** is a **local network**, e.g., one room, or one floor, or even one big building.
  - We say LAN as long as computers are close to each other.
  - You can also define a LAN when all computers are Ethernet connected.
  - A LAN can contain multiple smaller LANs.
- A **MAN** is something in between a LAN and a WAN, e.g., comprising several buildings on the same campus or in the same city. A MAN can use **FDDI** or **Ethernet** or other protocols for connectivity.



Lan-Wan-Man

- A **WAN** is a network with a lot of distance between the computers (or hosts).
  - A WAN does not use Ethernet, but protocols like **FDDI**, frame relay, **ATM** or **X.25** to connect computers (and networks).
- **PAN-WPAN**
  - Your home network is called a PAN (Personal Area Network).
  - A wireless PAN is a WPAN.



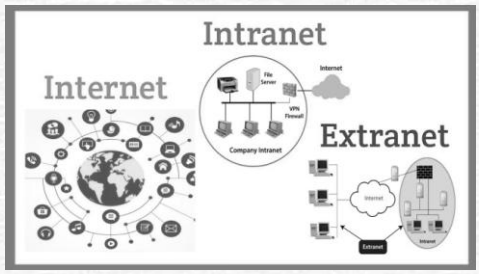
13

Internet - Intranet - Extranet

14

Internet - Intranet - Extranet

- The **Internet** is a global network. It connects many networks using the tcp/ip protocol stack.
- An **Intranet** is a private tcp/ip network. An Intranet uses the same protocols as the Internet, but is only accessible to people from within **one organization**.
- An **Extranet** is similar to an **Intranet**, but some trusted organizations partners/clients/ suppliers/...) also get access.



15

TCP/IP

16

TCP/IP

- The protocols that are used on the internet are defined in **rfc's**.
  - A **request for comment** describes the inner working of **Internet protocols**.
  - The **IETF** (Internet Engineering Task Force) is the sole publisher of these protocols since 1986.
  - The official website for the rfc's is http://www.rfc-editor.org. This website contains all rfc's in plain text.
  - For reliable connections, you use **tcp**, whereas **udp** is connectionless but faster. The **icmp** error messages are used by **ping**, multicast groups are managed by **igmp**.
    - ✓ Network **cards** are uniquely identified by their **MAC** address, **hosts** by their **IP** address and **applications** by their **port** number.
    - ✓ Common application level protocols like smtp, http, ssh, telnet and ftp have fixed port numbers. There is a list of port numbers in /etc/services.

```
bakhouya@ubuntu:~$ grep tcp /etc/protocols
tcp       6       TCP             # transmission control protocol
bakhouya@ubuntu:~$ grep ssh /etc/services
ssh             22/tcp                          # SSH Remote Login Protocol
bakhouya@ubuntu:~$ grep http /etc/services
# Updated from https://www.iana.org/assignments/service-names-port-numbers/servi
ce-names-port-numbers.xhtml .
http            80/tcp          www             # WorldWideWeb HTTP
https           443/tcp                         # http protocol over TLS/SSL
http-alt        8080/tcp        webcache        # WWW caching service
bakhouya@ubuntu:~$
```

17

Network configuration using Namespaces

18

interface configuration

- Configure network interface cards to work with tcp/ip (**1_NetConf**).
  - ip addr # show IP addresses of interfaces
  - ip route # show routing table
  - ip link # show network interfaces
  - ping 192.168.10.1 # Test connectivity
  - cat /etc/netplan/*.yaml

```
PC1-mini-shell> vi /etc/netplan/01-netcfg.yaml
PC1-mini-shell> ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
14: veth1@if13: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default qlen 1000
    link/ether d2:ca:a5:67:56:56 brd ff:ff:ff:ff:ff:ff link-netns GATEWAY
    inet 192.168.10.10/24 scope global veth1
       valid_lft forever preferred_lft forever
    inet6 fe80::d0ca:a5ff:fe67:5656/64 scope link
       valid_lft forever preferred_lft forever
PC1-mini-shell> ip route
192.168.10.0/24 dev veth1 proto kernel scope link src 192.168.10.10
192.168.20.0/24 via 192.168.10.1 dev veth1
PC1-mini-shell> ping 192.168.20.10
```

19

interface configuration

- Configure network interface cards to work with tcp/ip.
  - **/sbin/ifdown**: down an interface before changing its configuration.
  - **/sbin/ifup**: bringing the eth0 ethernet interface up.

```
PC1-mini-shell> ip link show veth1
14: veth1@if13: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP mode DEFAULT group default q
len 1000
    link/ether d2:ca:a5:67:56:56 brd ff:ff:ff:ff:ff:ff link-netns GATEWAY
PC1-mini-shell> ip link set veth1 down
PC1-mini-shell> ip link show veth1
14: veth1@if13: <BROADCAST,MULTICAST> mtu 1500 qdisc noqueue state DOWN mode DEFAULT group default qlen 1000
    link/ether d2:ca:a5:67:56:56 brd ff:ff:ff:ff:ff:ff link-netns GATEWAY
PC1-mini-shell> ip route
PC1-mini-shell> ip link set veth1 up
PC1-mini-shell> ping 192.168.20.10
ping: connect: Network is unreachable
PC1-mini-shell> ip route
192.168.10.0/24 dev veth1 proto kernel scope link src 192.168.10.10
PC1-mini-shell> ip route add 192.168.20.0/24 via 192.168.10.1
PC1-mini-shell> ping 192.168.20.10
PING 192.168.20.10 (192.168.20.10) 56(84) bytes of data.
64 bytes from 192.168.20.10: icmp_seq=1 ttl=63 time=0.037 ms
^C
--- 192.168.20.10 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.037/0.037/0.037/0.000 ms
PC1-mini-shell>
```

20

interface configuration

- The configuration file, **/etc/netplan/*.yaml**, is for network interfaces , used to define IP addresses, gateways, DNS, and interface settings (**sudo netplan apply command to apply changes**).

```
network:
  version: 2
  renderer: NetworkManager
  ethernets:
    eth0:
      dhcp4: yes
```

- **nmcli** is a command-line interface for **NetworkManager**. It reads/controls network configurations if the **netplan** YAML uses **renderer: NetworkManager**.
  - If the netplan YAML uses **renderer** "NetworkManager", you can use nmcli instead of editing YAML to configure interfaces. If it uses networkd, nmcli has no effect.
    - ✓ nmcli device status
    - ✓ nmcli connection show

interface configuration

- **Ifconfig**
  - **Ifconfig :** without any arguments will present you with a list of all **active network interface cards**, including **wireless** and the **loopback** interface.
  - **ifconfig eth0**: obtain information about just one **network card**.
  - **ifconfig eth0 down** or **up**: bring an interface **up** or **down**.
  - The ifconfig tool is deprecated on some systems. Use the **ip tool** instead.

- **Setting ip address**: temporary set an ip address with ifconfig. This ip address is only valid until the next ifup/ifdown cycle or until the next reboot.

```
PC1-mini-shell> ifconfig veth1 | grep 192
        inet 192.168.10.10  netmask 255.255.0.0  broadcast 192.168.255.255
PC1-mini-shell> ifconfig veth1 192.168.10.11 netmask 255.255.0.0
PC1-mini-shell> ifconfig veth1 | grep 192
        inet 192.168.10.11  netmask 255.255.0.0  broadcast 192.168.255.255
PC1-mini-shell>
```

Network sniffing

- A network administrator should be able to use a **sniffer** like **wireshark** or **tcpdump** to troubleshoot network problems.

```
PC1-mini-shell> ping 192.168.20.10
PING 192.168.20.10 (192.168.20.10) 56(84) bytes of data.
64 bytes from 192.168.20.10: icmp_seq=1 ttl=63 time=0.034 ms
64 bytes from 192.168.20.10: icmp_seq=2 ttl=63 time=0.032 ms
^C
--- 192.168.20.10 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1011ms
rtt min/avg/max/mdev = 0.032/0.033/0.034/0.001 ms
PC1-mini-shell>
```

```
PC2-mini-shell> sudo ip netns exec PC2 tcpdump -i veth2 -n
tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on veth2, link-type EN10MB (Ethernet), capture size 262144 bytes
17:10:47.813724 IP 192.168.10.11 > 192.168.20.10: ICMP echo request, id 1225, se
q 1, length 64
17:10:47.813737 IP 192.168.20.10 > 192.168.10.11: ICMP echo reply, id 1225, seq
1, length 64
17:10:48.824895 IP 192.168.10.11 > 192.168.20.10: ICMP echo request, id 1225, se
q 2, length 64
17:10:48.824908 IP 192.168.20.10 > 192.168.10.11: ICMP echo reply, id 1225, seq
```


Introduction to iptables

- **Iptables** is the Linux **firewall** and **packet-filtering** tool.
- **It controls:**
  - **Security** (allow / block traffic)
  - **NAT** (SNAT, DNAT, port forwarding)
  - **Packet forwarding** (routing through a gateway)
  - iptables architecture (simple): Table → Chain → Rule → Action

Plan

- Revisiting Network Basics
  - Network layers
  - Unicast, Multicast, Broadcast, Anycast
  - Lan-Wan-Man
  - Internet - Intranet - Extranet
  - Tcp/Ip
- **Introduction to routers**
  - **Router or firewall**
  - **iptables Firewall**
  - **iptables NAT**

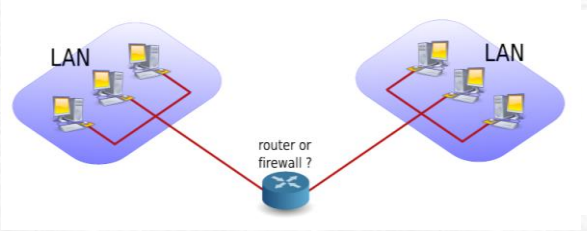
25

Router or firewall

26

Router or firewall

- A **router** is a **device** that **connects** two networks.
- A **firewall** is a device that besides acting as a router, also implements rules to determine whether packets are allowed to travel from one network to another.
  - A **firewall** can be configured to **block access** based on networks, hosts, protocols and ports.
  - Firewalls can also **change the contents** of packets while forwarding them.



27

Router or firewall

- **Packet forwarding**
  - Allowing packets to go from one network to another.
  - When a multihomed host is connected to two different networks, and it allows packets to travel from one network to another through its two network interfaces, it is said to have **enabled packet forwarding**.
- **Packet filtering**
  - Very similar to packet forwarding, but every packet is **individually tested** against **rules** that decide on allowing or dropping the packet. The rules are stored by **iptables**.
- **NAT(network address translation)**
  - A **NAT** device is a router that is also **changing** the **source** and/or **target** ip-**address** in packets.
  - It is typically used to **connect** multiple **computers** in a **private** address range with the (**public**) internet.
  - A NAT can **hide private** addresses from the **internet**.

28

Router or firewall

- **PAT(port address translation)**
  - A PAT device is a router that is also **changing** the **source** and/or target tcp/udp port in packets.
  - It is Cisco terminology and is used by SNAT, DNAT, masquerading and port forwarding.
- **SNAT(source NAT)**
  - A SNAT device is changing the **source ip-address** when a packet passes the NAT.
    - SNAT configuration with iptables includes a fixed target source address.
- **Masquerading**
  - A form of SNAT that will **hide the (private) source ip-addresses** of your private network using a **public ip-address**.
  - Masquerading is common on dynamic internet interfaces (broadband modem/routers).
  - Masquerade configuration with iptables uses a dynamic target source address.
- DNAT(destination NAT)
  - A DNAT device is changing the **destination ip-address** when a packet passes the NAT.

29

Router or firewall

- **Port forwarding**
  - When static DNAT is set up in a way that allows outside connections to enter our private network, then we call **it port forwarding**.
- Whether a host is forwarding packets is defined in **/proc/sys/net/ipv4/ip_forward**.
  - To enable packet forwarding: **echo 1 > /proc/sys/net/ipv4/ip_forward**
  - To check if packet forwarding is enabled: **cat /proc/sys/net/ipv4/ip_forward**
  - To disable packet forwarding: **echo 0 > /proc/sys/net/ipv4/ip_forward**

```
PC1-mini-shell> cat /proc/sys/net/ipv4/ip_forward
1
PC1-mini-shell> echo 0 > /proc/sys/net/ipv4/ip_forward
PC1-mini-shell> cat /proc/sys/net/ipv4/ip_forward
0
PC1-mini-shell> echo 1 > /proc/sys/net/ipv4/ip_forward
PC1-mini-shell>
```

- To enable packet forwarding whenever the **system starts**, change the **net.ipv4.ip_forward** variable in **/etc/sysctl.conf** to the value 1.

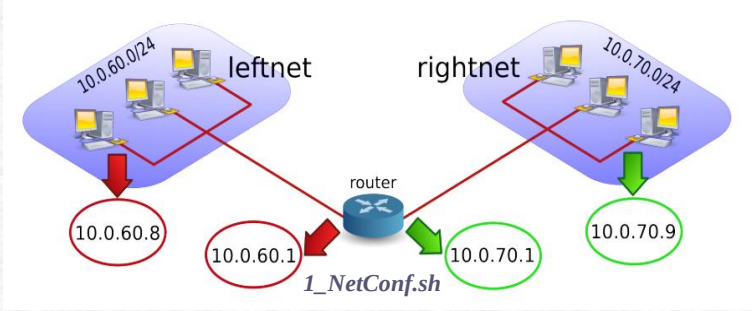
```
# Uncomment the next line to enable packet forwarding for IPv4
#net.ipv4.ip_forward=1
```

```
PC1-mini-shell> sysctl -a 2>/dev/null | grep ip_forward
net.ipv4.ip_forward = 1
net.ipv4.ip_forward_update_priority = 1
net.ipv4.ip_forward_use_pmtu = 0
PC1-mini-shell>
```

30

Router or firewall

- Set up two LANs, one on **leftnet**, the other on **rightnet** (**1_NetConf**)



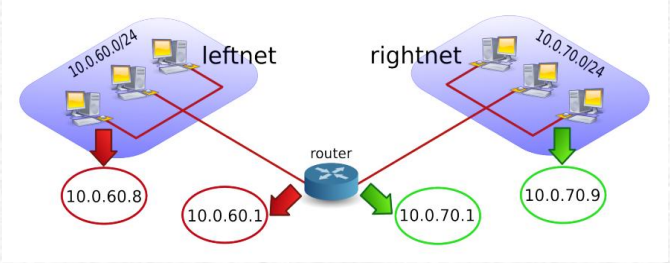
- Create **namespaces** and **veth** pairs to connect namespaces

```
bakhouya@ubuntu:~/Documents$ sudo ip netns add Cl
bakhouya@ubuntu:~/Documents$ sudo ip netns add Cr
bakhouya@ubuntu:~/Documents$ sudo ip netns add R
bakhouya@ubuntu:~/Documents$ sudo ip link add vethl type veth peer name Rl
bakhouya@ubuntu:~/Documents$ sudo ip link add vethr type veth peer name Rr
bakhouya@ubuntu:~/Documents$ ip netns list
R
Cr
Cl
bakhouya@ubuntu:~/Documents$
```

31

Router or firewall

- Set up two LANs, one on **leftnet**, the other on **rightnet**.



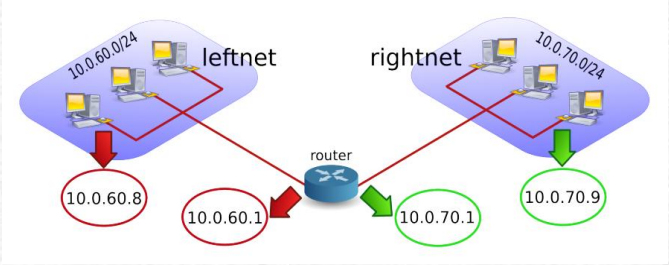
- Assign veth ends to namespaces

```
bakhouya@ubuntu:~/Documents$ sudo ip link set vethl netns Cl
bakhouya@ubuntu:~/Documents$ sudo ip link set vethr netns Cr
bakhouya@ubuntu:~/Documents$ sudo ip link set Rl netns R
bakhouya@ubuntu:~/Documents$ sudo ip link set Rr netns R
bakhouya@ubuntu:~/Documents$
```

32

Router or firewall

- Set up two LANs, one on **leftnet**, the other on **rightnet**.



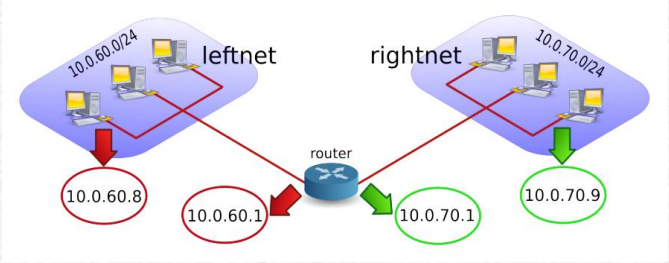
- Bring up interfaces inside namespaces

```
bakhouya@ubuntu:~/Documents$ sudo ip netns exec Cl ip addr add 10.0.70.9/24 dev vethl
bakhouya@ubuntu:~/Documents$ sudo ip netns exec Cr ip addr add 10.0.60.8/24 dev vethr
bakhouya@ubuntu:~/Documents$ sudo ip netns exec Cl ip link set vethl up
bakhouya@ubuntu:~/Documents$ sudo ip netns exec Cr ip link set vethr up
bakhouya@ubuntu:~/Documents$ sudo ip netns exec Cl ip link set lo up
bakhouya@ubuntu:~/Documents$ sudo ip netns exec Cr ip link set lo up
bakhouya@ubuntu:~/Documents$
```

33

Router or firewall

- Set up two LANs, one on **leftnet**, the other on **rightnet**.



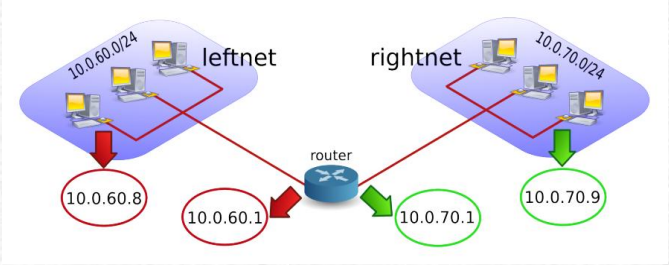
- Bring up interfaces inside namespaces

```
bakhouya@ubuntu:~/Documents$ sudo ip netns exec R ip addr add 10.0.70.1/24 dev Rl
bakhouya@ubuntu:~/Documents$ sudo ip netns exec R ip addr add 10.0.60.1/24 dev Rr
bakhouya@ubuntu:~/Documents$ sudo ip netns exec R ip link set Rl up
bakhouya@ubuntu:~/Documents$ sudo ip netns exec R ip link set Rr up
bakhouya@ubuntu:~/Documents$ sudo ip netns exec R ip link set lo up
bakhouya@ubuntu:~/Documents$
```

34

Router or firewall

- Set up two LANs, one on **leftnet**, the other on **rightnet**.



- Add routes

```
bakhouya@ubuntu:~/Documents$ sudo ip netns exec Cl ip route add default via 10.0.70.1
bakhouya@ubuntu:~/Documents$ sudo ip netns exec Cr ip route add default via 10.0.60.1
bakhouya@ubuntu:~/Documents$ sudo ip netns exec R sysctl -w net.ipv4.ip_forward=1
net.ipv4.ip_forward = 1
bakhouya@ubuntu:~/Documents$
```


Router or firewall

- Set up two LANs, one on **leftnet**, the other on **rightnet**.

```
bakhouya@ubuntu:~/Documents$ sudo ip netns exec Cl ip link
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN mode DEFAULT group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
18: vethl@if17: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP mode DEFAULT group defaul
t qlen 1000
    link/ether ee:b9:5d:71:ba:a0 brd ff:ff:ff:ff:ff:ff link-netns R
bakhouya@ubuntu:~/Documents$ sudo ip netns exec Cl ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
18: vethl@if17: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default qlen 1000
    link/ether ee:b9:5d:71:ba:a0 brd ff:ff:ff:ff:ff:ff link-netns R
    inet 10.0.70.9/24 scope global vethl
       valid_lft forever preferred_lft forever
    inet6 fe80::ecb9:5dff:fe71:baa0/64 scope link
       valid_lft forever preferred_lft forever
bakhouya@ubuntu:~/Documents$
```

Router or firewall

- Enable and/or disable packet forwarding on the router and verify what happens to the ping between the two networks.

```
bakhouya@ubuntu:~/Documents$ sudo ip netns exec R sysctl -w net.ipv4.ip_forward=1
net.ipv4.ip_forward = 1
bakhouya@ubuntu:~/Documents$ sudo ip netns exec Cl ping 10.0.60.8
PING 10.0.60.8 (10.0.60.8) 56(84) bytes of data.
64 bytes from 10.0.60.8: icmp_seq=1 ttl=63 time=0.029 ms
^C
--- 10.0.60.8 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.029/0.029/0.029/0.000 ms
bakhouya@ubuntu:~/Documents$ sudo ip netns exec R sysctl -w net.ipv4.ip_forward=0
net.ipv4.ip_forward = 0
bakhouya@ubuntu:~/Documents$ sudo ip netns exec Cl ping 10.0.60.8
PING 10.0.60.8 (10.0.60.8) 56(84) bytes of data.
^C
--- 10.0.60.8 ping statistics ---
2 packets transmitted, 0 received, 100% packet loss, time 1015ms

bakhouya@ubuntu:~/Documents$
```

37

Router or firewall

- Use **arp -a** to make sure you are connected with the correct mac addresses.

```
bakhouya@ubuntu:~/Documents$ sudo ip netns exec R arp -a
? (10.0.70.9) at ee:b9:5d:71:ba:a0 [ether] on Rl
? (10.0.60.8) at 56:d9:64:d5:d3:91 [ether] on Rr
bakhouya@ubuntu:~/Documents$ sudo ip netns exec Cl arp -a
? (10.0.70.1) at 62:91:33:46:bb:db [ether] on vethl
bakhouya@ubuntu:~/Documents$ sudo ip netns exec Cr arp -a
? (10.0.60.1) at da:7e:ed:26:f2:d4 [ether] on vethr
bakhouya@ubuntu:~/Documents$
```

	Cl	Rl		Cr
MAC	56:d9:64:d5:d3:91	da:7e:ed:26:f2:d4	62:91:33:46:bb:db	Ee:b9:5d:71:ba:a0
IP	192.168.60.8	192.168.60.1	192.168.70.1	192.168.70.9

- Observe how **MAC** and **IP** addresses change when a packet passes through the **router**.
  - Capture traffic on left LAN (Cl → Router)
  - Capture traffic on right LAN (Router→ Cr)
  - Cl ping Cr (sudo ip netns exec Cl ping -c 2 10.0.60.8)

38

Router or firewall

	Cr	Rl		Cl
MAC	56:d9:64:d5:d3:91	da:7e:ed:26:f2:d4	62:91:33:46:bb:db	Ee:b9:5d:71:ba:a0
IP	192.168.60.8	192.168.60.1	192.168.70.1	192.168.70.9

```
bakhouya@ubuntu:~/Documents$ sudo ip netns exec Cl ping -c 2 10.0.60.8
PING 10.0.60.8 (10.0.60.8) 56(84) bytes of data.
64 bytes from 10.0.60.8: icmp_seq=1 ttl=63 time=0.034 ms
64 bytes from 10.0.60.8: icmp_seq=2 ttl=63 time=0.033 ms
```

```
bakhouya@ubuntu:~/Documents$ sudo ip netns exec R tcpdump -xx -i Rr
tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on Rr, link-type EN10MB (Ethernet), capture size 262144 bytes
23:53:26.116676 IP 10.0.70.9 > 10.0.60.8: ICMP echo request, id 61751, seq 1, length 64
        0x0000:  56d9 64d5 d391 da7e ed26 f2d4 0800 4500
```

```
23:53:26.116686 IP 10.0.60.8 > 10.0.70.9: ICMP echo reply, id 61751, seq 1, length 64
        0x0000:  da7e ed26 f2d4 56d9 64d5 d391 0800 4500
```

```
bakhouya@ubuntu:~/Documents$ sudo ip netns exec R tcpdump -xx -i Rl
tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on Rl, link-type EN10MB (Ethernet), capture size 262144 bytes
23:53:26.116664 IP 10.0.70.9 > 10.0.60.8: ICMP echo request, id 61751, seq 1, length 64
        0x0000:  6291 3346 bbdb eeb9 5d71 baa0 0800 4500
```

```
23:53:26.116688 IP 10.0.60.8 > 10.0.70.9: ICMP echo reply, id 61751, seq 1, length 64
        0x0000:  eeb9 5d71 baa0 6291 3346 bbdb 0800 4500
```

39

Router or firewall

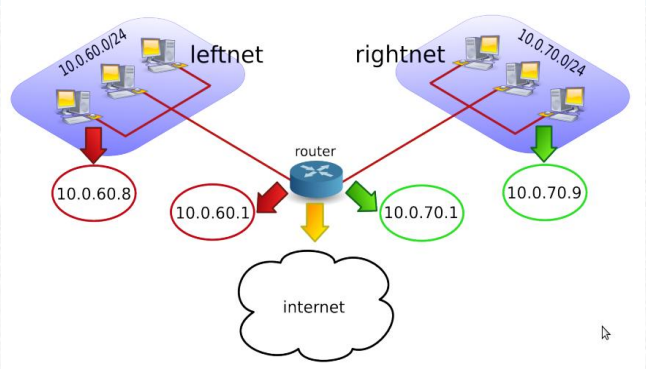
	Cr	Rl		Cl
MAC	56:d9:64:d5:d3:91	da:7e:ed:26:f2:d4	62:91:33:46:bb:db	Ee:b9:5d:71:ba:a0
IP	192.168.60.8	192.168.60.1	192.168.70.1	192.168.70.9

```
23:53:31.170061 ARP, Request who-has 10.0.60.8 tell 10.0.60.1, length 28
        0x0000:  56d9 64d5 d391 da7e ed26 f2d4 0806 0001
        0x0010:  0800 0604 0001 da7e ed26 f2d4 0a00 3c01
        0x0020:  0000 0000 0000 0a00 3c08
23:53:31.170077 ARP, Request who-has 10.0.60.1 tell 10.0.60.8, length 28
        0x0000:  da7e ed26 f2d4 56d9 64d5 d391 0806 0001
        0x0010:  0800 0604 0001 56d9 64d5 d391 0a00 3c08
        0x0020:  0000 0000 0000 0a00 3c01
23:53:31.170080 ARP, Reply 10.0.60.1 is-at da:7e:ed:26:f2:d4 (oui Unknown), length 28
        0x0000:  56d9 64d5 d391 da7e ed26 f2d4 0806 0001
        0x0010:  0800 0604 0002 da7e ed26 f2d4 0a00 3c01
        0x0020:  56d9 64d5 d391 0a00 3c08
23:53:31.170088 ARP, Reply 10.0.60.8 is-at 56:d9:64:d5:d3:91 (oui Unknown), length 28
        0x0000:  da7e ed26 f2d4 56d9 64d5 d391 0806 0001
        0x0010:  0800 0604 0002 56d9 64d5 d391 0a00 3c08
        0x0020:  da7e ed26 f2d4 0a00 3c01
^C
8 packets captured
8 packets received by filter
0 packets dropped by kernel
```

40

Router or firewall

	Cr	Rl		Cl
MAC	56:d9:64:d5:d3:91	da:7e:ed:26:f2:d4	62:91:33:46:bb:db	Ee:b9:5d:71:ba:a0
IP	192.168.60.8	192.168.60.1	192.168.70.1	192.168.70.9

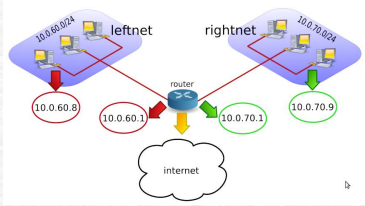


- **Give internet access to leftnet and rightnet.**

41

Router or firewall

- Namespaces:
  - Cl → Left LAN (leftnet) → 10.0.70.9
  - Cr → Right LAN (rightnet) → 10.0.60.8
  - R → Router namespace, connecting both LANs and the Internet
- Interfaces on Router R:
  - Rl → connects to Right LAN → 10.0.60.1
  - Rr → connects to Left LAN → 10.0.70.1
  - Ri → connects to Internet → public IP assigned
- LANs:
  - Leftnet: 10.0.70.0/24
  - Rightnet: 10.0.60.0/24
- Internet:
  - Simulated ISP namespace → 209.165.200.230 (example public IP)
- Routing and NAT Concept
  - R will act as the default gateway for both LANs:
    - ✓ Left LAN (Cl) → default gateway: 10.0.70.1
    - ✓ Right LAN (Cr) → default gateway: 10.0.60.1
  - Static routes or NAT must be configured on R, both LANs can get to Internet
  - Packet flow: Cl sends a packet to the Internet → reaches R via Rr → R performs NAT → forwards to ISP; ISP → R NAT reverses → sends to Cl

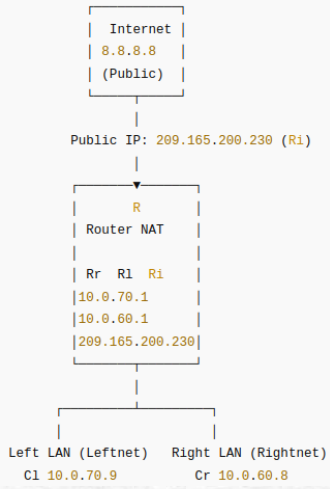


42

Router or firewall

- Simulate Internet access just with **R** and its **public IP** (209.165.200.230) without creating a full ISP namespace (Lab0).

NS	Inter.	IP Address	Role
Cl	eth0	10.0.70.9/24	Left LAN
Cr	eth0	10.0.60.8/24	Right LAN
R	Rr	10.0.70.1/24	Left LAN gateway
	Rl	10.0.60.1/24	Right LAN gateway
	Ri	209.165.200.230	Public interface (NAT)
Host	Ri-peer	209.165.200.229	Simulated ISP / ext hop



- Requirements
  - Cl ↔ Cr can ping each other □
  - Cl/Cr → R → public IP (209.165.200.230) □
  - NAT MASQUERADE is working

43

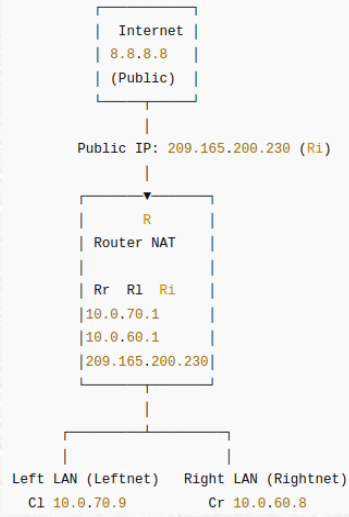
Router or firewall

- Verify internal LAN
  - **sudo ip netns exec Cl ping -c 3 10.0.60.8 # Cl → Cr**
  - **sudo ip netns exec Cr ping -c 3 10.0.70.9 # Cr → Cl**
- Create veth pair: Ri (Router) ↔ Ri-peer (Host/ISP)
  - **sudo ip link add Ri type veth peer name Ri-peer**
- Move Ri into R namespace
  - **sudo ip link set Ri netns R**
- Assign IPs inside Router R
  - **sudo ip netns exec R ip addr add 209.165.200.230/27 dev Ri**
  - **sudo ip netns exec R ip link set Ri up**
- Assign IPs on Host (acts as ISP)
  - **sudo ip addr add 209.165.200.229/27 dev Ri-peer**
  - **sudo ip link set Ri-peer up**
- Enable IP forwarding on Router R to allows R to forward packets between LANs and Ri.
  - **sudo ip netns exec R sysctl -w net.ipv4.ip_forward=1**

44

Router or firewall

- Flow Example: Cl → Internet (**2_add_nat_internet.sh**)
  - Cl → R
    - ✓ Source IP: 10.0.70.9
    - ✓ Destination: 8.8.8.8
  - R NAT (MASQUERADE)
    - ✓ Source IP replaced with 209.165.200.230
    - ✓ Destination stays 8.8.8.8
  - Internet sees packet
    - ✓ Src: 209.165.200.230
    - ✓ Dst: 8.8.8.8
  - Reply comes back to R
    - ✓ NAT table maps reply to 10.0.70.9
  - R → Cl
    - ✓ Src: 8.8.8.8
    - ✓ Dst: 10.0.70.9



45

Router or firewall

- Configure **NAT** on R (for both LANs), let packets leaving R's public interface (**Ri**) appear as coming from **209.165.200.230**, i.e., NAT for Left/right LANs → public IP Ri:
  - sudo ip netns exec R iptables -t nat -A POSTROUTING -s 10.0.70.0/24 -o Ri -j MASQUERADE
  - sudo ip netns exec R iptables -t nat -A POSTROUTING -s 10.0.60.0/24 -o Ri -j MASQUERADE
- Set a default route on R (to "Internet"), send everything unknown to the host Internet
  - sudo ip netns exec R ip route add default dev Ri

```
bakhouya@ubuntu:~/Documents/Cours$ sudo ip link add Ri type veth peer name Ri-peer
bakhouya@ubuntu:~/Documents/Cours$ sudo ip link set Ri netns R
bakhouya@ubuntu:~/Documents/Cours$ sudo ip netns exec R ip addr add 209.165.200.230/27 dev Ri
bakhouya@ubuntu:~/Documents/Cours$ sudo ip netns exec R ip link set Ri up
bakhouya@ubuntu:~/Documents/Cours$ sudo ip addr add 209.165.200.229/27 dev Ri-peer
bakhouya@ubuntu:~/Documents/Cours$ sudo ip link set Ri-peer up
bakhouya@ubuntu:~/Documents/Cours$ sudo ip netns exec R sysctl -w net.ipv4.ip_forward=1
net.ipv4.ip_forward = 1
bakhouya@ubuntu:~/Documents/Cours$ sudo ip netns exec R iptables -t nat -A POSTROUTING -s 10.0.70.0/24 -o Ri -j MASQUERADE
bakhouya@ubuntu:~/Documents/Cours$ sudo ip netns exec R iptables -t nat -A POSTROUTING -s 10.0.60.0/24 -o Ri -j MASQUERADE
bakhouya@ubuntu:~/Documents/Cours$ sudo ip netns exec R ip route add default via 209.165.200.229 dev Ri
```

```
bakhouya@ubuntu:~/Documents/Cours$ sudo ip netns exec Cl ping -c 3 209.165.200.230
PING 209.165.200.230 (209.165.200.230) 56(84) bytes of data.
64 bytes from 209.165.200.230: icmp_seq=1 ttl=64 time=0.042 ms
^C
```

46

iptables firewall

iptables firewall

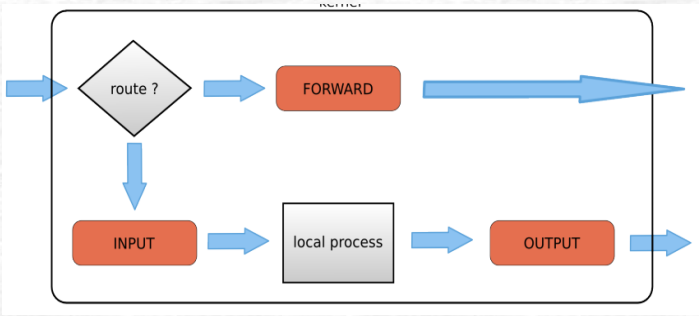
- **iptables** is an application that allows a user to configure the firewall functionality.
- By default there are two tables that contain sets of rules.
  - The **filter table** is used for **packet filtering**.

```
bakhouya@ubuntu:~/Documents/Cours/2$ sudo ip netns exec R iptables -t filter -L
Chain INPUT (policy ACCEPT)
target     prot opt source               destination

Chain FORWARD (policy ACCEPT)
target     prot opt source               destination

Chain OUTPUT (policy ACCEPT)
target     prot opt source               destination
```

  - **Packet filtering** is a bit more than **packet forwarding**.
    - Packet forwarding uses only a routing table to make decisions.
    - Packet filtering uses a list of rules to figure out what to do with each packet.

iptables firewall

- The filter table in **iptables** has **three chains** (sets of rules).
  - **INPUT**: traffic to the router itself (management, monitoring, etc.).
  - **OUTPUT**: traffic from the router itself (logs, pings, updates).
  - **FORWARD**: traffic passing through the router between two networks.



49

iptables firewall

- List the filter table and all its rules. All three chains in the filter table are set to **ACCEPT** everything. ACCEPT is the **default behaviour**.
  - **t filter**: firewall table (default, but explicit is clearer)
  - **L**: list rules
  - **v** : packets & bytes counters
  - **n** : no DNS lookup (faster, cleaner)

```
bakhouya@ubuntu:~/Documents/Cours$ sudo ip netns exec R iptables -t filter -L -v -n
Chain INPUT (policy ACCEPT 8 packets, 1972 bytes)
 pkts bytes target     prot opt in     out     source               destination

Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target     prot opt in     out     source               destination

Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target     prot opt in     out     source               destination
bakhouya@ubuntu:~/Documents/Cours$
```

50

iptables firewall

- Setting default rules
  - A more secure setup would be to **DROP** everything.

```
bakhouya@ubuntu:~/Documents/Cours$ sudo ip netns exec R iptables -P INPUT DROP
bakhouya@ubuntu:~/Documents/Cours$ sudo ip netns exec R iptables -P OUTPUT DROP
bakhouya@ubuntu:~/Documents/Cours$ sudo ip netns exec R iptables -P FORWARD DROP
bakhouya@ubuntu:~/Documents/Cours$ sudo ip netns exec R iptables -L
Chain INPUT (policy DROP)
target     prot opt source               destination

Chain FORWARD (policy DROP)
target     prot opt source               destination

Chain OUTPUT (policy DROP)
target     prot opt source               destination
bakhouya@ubuntu:~/Documents/Cours$
```

- Flush filter rules (remove rules, keep policies)
  - **F** removes all rules in the chain, default policy is NOT changed

```
bakhouya@ubuntu:~$ sudo ip netns exec R iptables -F INPUT
bakhouya@ubuntu:~$ sudo ip netns exec R iptables -F OUTPUT
bakhouya@ubuntu:~$ sudo ip netns exec R iptables -F FORWARD
bakhouya@ubuntu:~$
```

51

iptables firewall: INPUT

- When is INPUT used (**Traffic TO the router**) ?
  - When the destination IP is the router itself, e.g., block ping to router.
    - ✓ Packet ends at R; Firewall checks INPUT; Rule matches DROP
- Example for securing the router using basic iptables rules (
  - Block ping to the router (**3_test_block_ping_router**)
    - ✓ INPUT → traffic to router
    - ✓ ICMP type 8 = echo request (ping)
    - ✓ DROP → silently discard packet
    - ✓ Router becomes invisible to ping
    - ✓ sudo ip netns exec R iptables -A INPUT -p icmp --icmp-type 8 -j DROP
  - Test from Cl
    - ✓ sudo ip netns exec Cl ping -c 3 10.0.70.1

```
bakhouya@ubuntu:~$ sudo ip netns exec R iptables -A INPUT -p icmp --icmp-type 8 -j DROP
bakhouya@ubuntu:~$ sudo ip netns exec R iptables -L INPUT -v -n --line-numbers
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
num   pkts bytes target     prot opt in     out     source               destination
1        0     0 DROP       icmp --  *      *       0.0.0.0/0            0.0.0.0/0           icmptype 8
bakhouya@ubuntu:~$ sudo ip netns exec Cl ping -c 3 10.0.70.1
PING 10.0.70.1 (10.0.70.1) 56(84) bytes of data.
^C
--- 10.0.70.1 ping statistics ---
3 packets transmitted, 0 received, 100% packet loss, time 2050ms
```

52

iptables firewall: FORWARD

- FORWARD CHAIN (Traffic THROUGH the router)
  - The FORWARD is used when the router is NOT the destination, only passing traffic along, e.g., Cl → R → Internet (**3_test_input_forward_ping**)
  - Example: Block ping to Internet
    - ✓ sudo ip netns exec R iptables -A FORWARD -p icmp --icmp-type 8 -j DROP

```
bakhouya@ubuntu:~$ sudo ip netns exec R iptables -A FORWARD -p icmp --icmp-type 8 -j DROP
[sudo] password for bakhouya:
bakhouya@ubuntu:~$ sudo ip netns exec R iptables -L FORWARD -v -n
Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target     prot opt in     out     source               destination
    0     0 DROP       icmp --  *      *       0.0.0.0/0            0.0.0.0/0           icmptype 8
bakhouya@ubuntu:~$ sudo ip netns exec Cl ping -c 2 209.165.200.230
PING 209.165.200.230 (209.165.200.230) 56(84) bytes of data.
64 bytes from 209.165.200.230: icmp_seq=1 ttl=64 time=0.030 ms
64 bytes from 209.165.200.230: icmp_seq=2 ttl=64 time=0.026 ms

--- 209.165.200.230 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1030ms
rtt min/avg/max/mdev = 0.026/0.028/0.030/0.002 ms
bakhouya@ubuntu:~$ sudo ip netns exec Cl ping -c 2 209.165.200.229
PING 209.165.200.229 (209.165.200.229) 56(84) bytes of data.
^C
--- 209.165.200.229 ping statistics ---
2 packets transmitted, 0 received, 100% packet loss, time 1033ms

bakhouya@ubuntu:~$
```

53

iptables firewall: OUTPUT

- OUTPUT chain (originating from R)
  - Any traffic **generated by the router itself** (like ping 8.8.8.8 from R)
  - Not used for traffic **passing through** (that's FORWARD)
  - Not used for traffic **destined to the router itself** (that's INPUT)

Packet Flow	Chain Hit
Cl → R	INPUT
Cl → Internet via R	FORWARD
R → Internet	OUTPUT

  - Block router's own ping
    - ✓ sudo ip netns exec R iptables -A OUTPUT -p icmp --icmp-type 8 -j DROP
    - ✓ sudo ip netns exec R ping -c 2 209.165.200.229

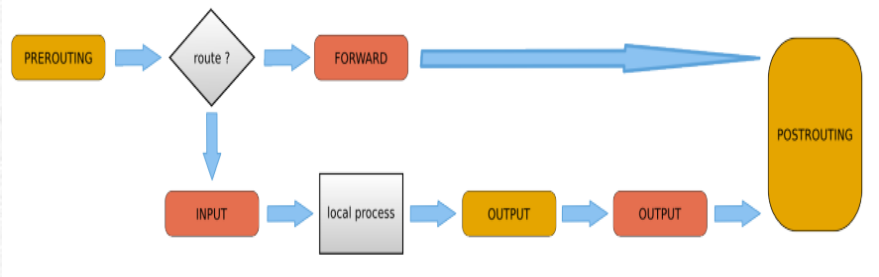
```
bakhouya@ubuntu:~$ sudo ip netns exec R ping -c 2 209.165.200.229
PING 209.165.200.229 (209.165.200.229) 56(84) bytes of data.
64 bytes from 209.165.200.229: icmp_seq=1 ttl=64 time=0.034 ms
64 bytes from 209.165.200.229: icmp_seq=2 ttl=64 time=0.031 ms

--- 209.165.200.229 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1026ms
rtt min/avg/max/mdev = 0.031/0.032/0.034/0.001 ms
bakhouya@ubuntu:~$ ^C
bakhouya@ubuntu:~$ sudo ip netns exec R iptables -A OUTPUT -p icmp --icmp-type 8 -j DROP
bakhouya@ubuntu:~$ sudo ip netns exec R ping -c 2 209.165.200.229
PING 209.165.200.229 (209.165.200.229) 56(84) bytes of data.
ping: sendmsg: Operation not permitted
ping: sendmsg: Operation not permitted
^C
--- 209.165.200.229 ping statistics ---
2 packets transmitted, 0 received, 100% packet loss, time 1032ms

bakhouya@ubuntu:~$
```

54

iptables NAT

NAT

- A router that is also changing the source and/or target ip-address in packets.
- Typically used to connect multiple computers in a private address range with the (public) internet.
- Can **hide private addresses** from the internet, disclose details about internal networks to the outside.
- The **NAT** table in **iptables** adds two new chains:
  - **PREROUTING**: allows altering of packets before they reach the INPUT chain.
  - **POSTROUTING**: allows altering packets after they exit the OUTPUT chain.



NAT

- A router that is also changing the source and/or target ip-address in packets.
- Typically used to connect multiple computers in a private address range with the (public) internet.

NAT Type	Direction	iptables target	Use case
SNAT	LAN → Internet	POSTROUTING	Change source IP (**MASQUERADE**)
DNAT	Internet → LAN	PREROUTING	Change destination IP (**expose server**)

Before SNAT
- Src: 10.0.70.9
- Dst: 209.165.200.229

After SNAT
- Src: 209.165.200.230
- Dst: 209.165.200.229

Incoming packet
- Src: Internet host
- Dst: 209.165.200.230

After DNAT
- Src: Internet host
- Dst: 192.168.70.2

NAT: SNAT

- Change the source address inside a packet before it leaves the system (e.g. to the internet, **2_add_nat_internet**).
- The destination will return the packet to the NAT-device, i.e., NAT-device will need to keep a table in memory of all the packets it changed, so it can deliver the packet to the original source (e.g. in the private network).
- SNAT is about packets leaving the system, it uses the POSTROUTING chain.

```
bakhouya@ubuntu:~$ sudo ip netns exec R iptables -t nat -nvL
[sudo] password for bakhouya:
Chain PREROUTING (policy ACCEPT 66 packets, 12348 bytes)
 pkts bytes target     prot opt in     out     source               destination

Chain INPUT (policy ACCEPT 10 packets, 1188 bytes)
 pkts bytes target     prot opt in     out     source               destination

Chain OUTPUT (policy ACCEPT 4 packets, 336 bytes)
 pkts bytes target     prot opt in     out     source               destination

Chain POSTROUTING (policy ACCEPT 2 packets, 168 bytes)
 pkts bytes target     prot opt in     out     source               destination
    4   336 MASQUERADE  all  --  *      Ri      10.0.70.0/24         0.0.0.0/0
    1    84 MASQUERADE  all  --  *      Ri      10.0.60.0/24         0.0.0.0/0
bakhouya@ubuntu:~$
```

NAT: SNAT

- In iptables, SNAT can be written as:
  - iptables -t nat -A POSTROUTING -s 10.0.70.0/24 -o Ri -j SNAT --to-source 209.165.200.230
  - sudo ip netns exec R iptables -t nat -A POSTROUTING -s 10.0.70.0/24 -o Ri -j MASQUERADE (our example, dynamic SNAT)
    - ✓ MASQUERADE automatically uses the router interface IP (Ri) as source.
    - ✓ Happens automatically, so you don't need to specify the router IP manually.
    - ✓ If therouter's public IP changes (like DHCP from ISP), MASQUERADE will automatically use the new IP.
    - ✓ Regular SNAT would require you to manually set the new source IP.

LAN Host (10.0.70.9) --> Router (Ri)
Original packet: Src=10.0.70.9, Dst=209.165.200.230
After MASQUERADE: Src=209.165.200.230, Dst=209.165.200.230
Internet sees packet: Src=209.165.200.230
Reply from Internet: Dst=209.165.200.230
Router automatically maps reply back to 10.0.70.9

59

NAT: DNAT

- Typically used to allow packets from the internet to be redirected to an internal server (in your DMZ) and in a private address range that is inaccessible directly form the internet.
  - Packets from the Internet are addressed to 209.165.200.230 (public IP)
  - Inside the LAN, the web server is 192.168.70.2, i.e., private IP not visible outside
  - Destination NAT (DNAT) changes the destination IP so packets reach the internal server
    - ✓ iptables -t nat -A PREROUTING -d 209.165.200.230 -p tcp --dport 80 -j DNAT --to-destination 192.168.70.2:80

-t nat	Operates on the NAT table
-A PREROUTING	Adds a rule, before routing
-d 209.165.200.230	Matches packets destined for the router's public IP
-p tcp --dport 80	Only matches TCP packets going to port 80 (HTTP)
-j DNAT --to-destination 192.168.70.2:80	rewrites the destination IP from 209.165.200.230 → 192.168.70.2

- ✓ See Lab related to Web server for more information about DNAT (**Lab3 and Lab5**).

60

References

- Paul Cobbaut, Linux Networking, http://linux-training.be/
- Linux Network Administrator's Guide, 2nd Edition, by Olaf Kirch & Terry Dawson; published by O'Reilly & Associates, Inc. 503 pages; https://www.linuxdoc.org/
- Jay LaCroix, Mastering Linux Network Administration, Packt Publishing Ltd. ISBN 978-1-78439-959-7, www.packtpub.com
- Remo Suppi Boldrito, Network administration, https://openaccess.uoc.edu/